

	Departamento: Departamento de Ciencias de la Computación.
	Carrera: Software Asignatura: Análisis y Diseño de Software NRC: 30724
	Apellidos y nombres del estudiante: • Lasso Sebastián • Valencia Yuliana • Villarroel Justin Taller N°2

ANTECEDENTES

En el desarrollo de software existen diferentes formas de organizar un sistema para que sea más fácil de entender, mantener y mejorar. Una de estas formas es mediante el uso de patrones de diseño, los cuales ayudan a resolver problemas comunes de programación.

Dentro de estos patrones se encuentran los patrones estructurales, que sirven para organizar las relaciones entre clases y objetos de una mejor manera. En este trabajo se estudiarán los patrones Composite, Proxy y Bridge, ya que cada uno cumple una función diferente dentro de un sistema.

El patrón Composite ayuda a trabajar con objetos individuales y grupos de objetos de la misma manera. El patrón Proxy permite controlar el acceso a ciertas partes del sistema usando un intermediario. Finalmente, el patrón Bridge ayuda a separar una parte del sistema de su implementación para que sea más flexible y fácil de modificar.

Estos patrones pueden aplicarse en el Sistema de Gestión de Mantenimientos para mejorar la organización del código y facilitar procesos críticos como la validación de perfiles, el procesamiento de reportes estadísticos y la integración con servicios externos de mensajería.

OBJETIVO

Analizar e implementar los patrones de diseño estructurales Composite, Proxy y Bridge en el desarrollo del Sistema de Gestión de Mantenimientos, con el fin de mejorar la organización, el control de acceso a la capa de negocio y la flexibilidad de la estructura del software ante integraciones externas.

DESARROLLO

1. Composite

En el módulo asociado al requisito RF-04: Gestionar Estadísticas, el sistema debe consolidar datos de volumen y tipos de mantenimiento en reportes estructurados.

- Implementación: Se puede crear una interfaz IComponenteMantenimiento. Los nodos hoja serán instancias de la clase Mantenimiento individual, y los nodos compuestos serán GrupoMantenimientos (por ejemplo, agrupaciones por mes, por técnico o por sucursal).

- Beneficio: Esto permite que el método procesarEstadisticas() del GestorCentral calcule totales, frecuencias y volúmenes de daños invocando un único método sobre la estructura, sin necesidad de escribir lógica condicional para distinguir si está iterando sobre un registro aislado o un bloque entero de datos históricos.

Implementación Composite

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

interface "IComponenteMantenimiento" as ICompMant <<entity>> {
+ obtenerCostoTotal(): double
}

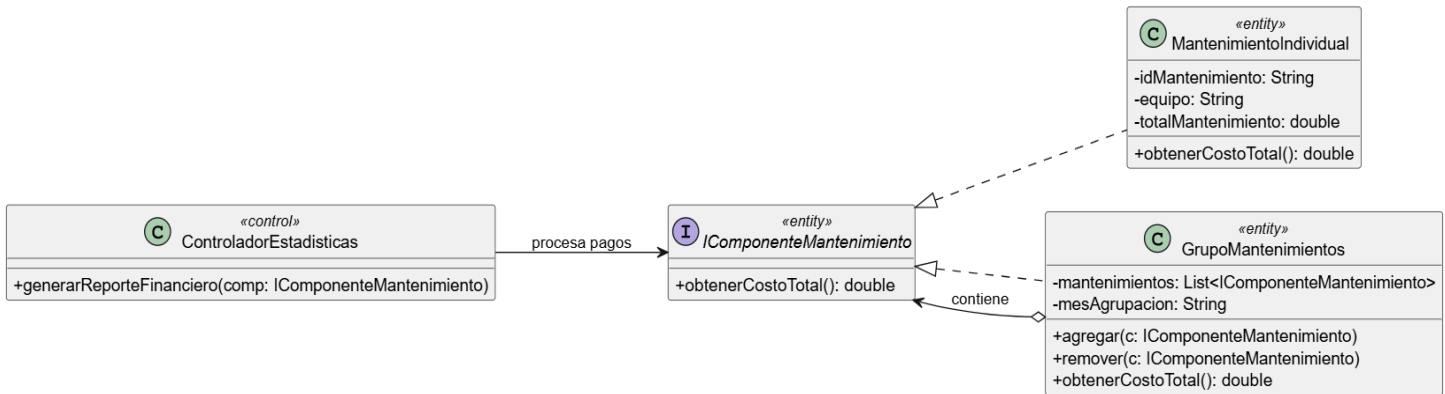
class "MantenimientoIndividual" as MantInd <<entity>> {
- idMantenimiento: String
- equipo: String
- totalMantenimiento: double
+ obtenerCostoTotal(): double
}

class "GrupoMantenimientos" as GrpMant <<entity>> {
- mantenimientos: List<IComponenteMantenimiento>
- mesAgrupacion: String
+ agregar(c: IComponenteMantenimiento)
+ remover(c: IComponenteMantenimiento)
+ obtenerCostoTotal(): double
}

class "ControladorEstadisticas" as CtrlEst <<control>> {
+ generarReporteFinanciero(comp: IComponenteMantenimiento)
}

ICompMant <|.. MantInd
ICompMant <|.. GrpMant
GrpMant o--> ICompMant : contiene

CtrlEst --> ICompMant : procesa pagos
@enduml
```



2. Proxy (Apoderado)

El sistema maneja tres actores distintos (Administrador, Técnico y Cliente) que interactúan a través de la misma frontera hacia el GestorCentral. Las reglas de negocio establecen diferencias claras de acceso; por ejemplo, el Administrador realiza el CRUD completo, pero el Técnico solo debe poder visualizar su propio perfil en modo lectura.

- Implementación: Se desarrolla la clase ProxyControlador que implementa la misma interfaz que el ControladoReal. La clase UI_ModuloGestion se comunicará con el Proxy.
- Beneficio: Antes de ejecutar métodos sensibles como gestionarCRUDUsuario() o guardarOActualizar(), el Proxy intercepta la petición, verifica el rol del actor logueado y determina si tiene los permisos necesarios. Si es un Técnico intentando borrar un registro, el Proxy bloquea la operación y registra el intento de acceso, garantizando la seguridad sin mezclar la lógica de validación con la lógica de negocio.

Implementación proxy

```

@startuml
skinparam classAttributelconSize 0
skinparam shadowing false
left to right direction

class "UI_SistemaMantenimiento" as UI <<boundary>> {
+ solicitarRegistroMantenimiento()
}

interface "IControladorPrincipal" as IControlador <<control>> {
+ registrarMantenimiento(datos: DTO): Resultado
}

class "ControladorReal" as CtrlReal <<control>> {

```

```
+ registrarMantenimiento(datos: DTO): Resultado
}
```

```
class "ProxyControlador" as Proxy <<control>> {
- controladorReal: ControladorReal
- sesionActiva: Usuario
+ registrarMantenimiento(datos: DTO): Resultado
- verificarPermisos(operacion: String): boolean
}
```

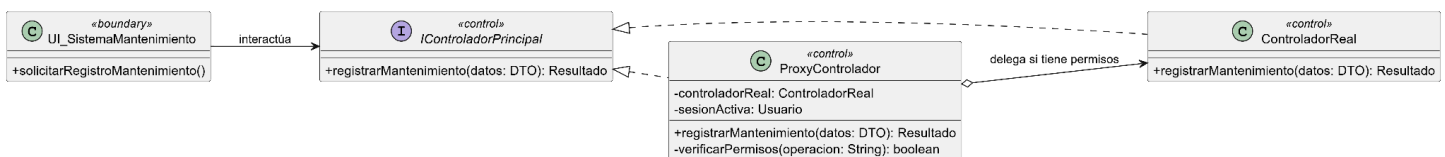
IControlador <|.. Proxy

IControlador <|.. CtrlReal

Proxy o--> CtrlReal : delega si tiene permisos

UI --> IControlador : interactúa

@enduml



3. Bridge (Puentes)

El caso de uso RF-03: Gestionar Mantenimientos requiere que el sistema envíe reportes detallados mediante APIs externas, específicamente WhatsApp y Correo.

- Implementación: En lugar de que el GestorCentral llame directamente a métodos acoplados, se crea una abstracción NotificadorReporte (que maneja qué información se envía) que mantiene una referencia a una interfaz puente IImplementadorAPI. De esta interfaz nacerán clases concretas como ImplementadorWhatsApp e ImplementadorCorreo.
- Beneficio: Permite separar la lógica de negocio (cuándo y a quién enviar el reporte) de los detalles de bajo nivel de la comunicación de red. Si a futuro la empresa requiere notificar vía SMS, solo se crea un nuevo implementador sin tocar el código del controlador principal.

Implementación Bridge

@startuml

skinparam classAttributeIconSize 0

skinparam shadowing false

left to right direction

' --- ABSTRACCIÓN ---

```
abstract class "NotificadorReporte" as Notificador <<control>> {
# proveedorAPI: IProveedorComunicaciones
+ setProveedor(p: IProveedorComunicaciones)
```

```

+ enviarReporteCliente(cliente: String, reporte: String)
}

class "NotificadorBasico" as NotifBasico <<control>> {
+ enviarReporteCliente(cliente: String, reporte: String)
}

' --- PUENTE ---
Notificador o--> IProveedorComunicaciones : usa puente

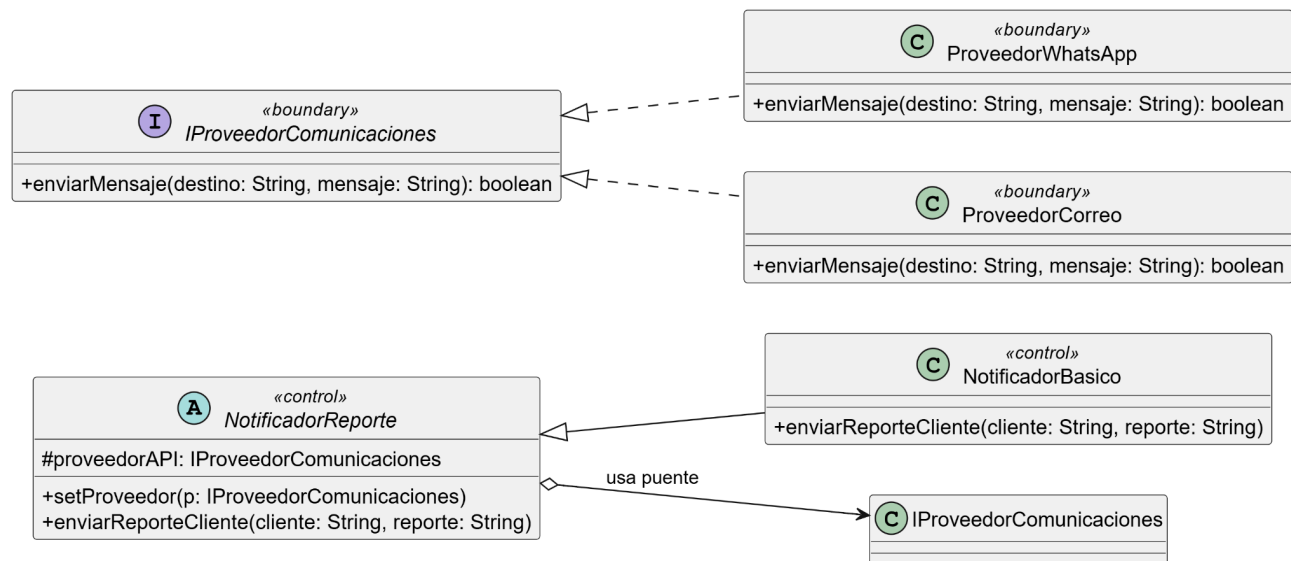
' --- IMPLEMENTACIÓN ---
interface "IProveedorComunicaciones" as IProvCom <<boundary>> {
+ enviarMensaje(destino: String, mensaje: String): boolean
}

class "ProveedorWhatsApp" as ProvWA <<boundary>> {
+ enviarMensaje(destino: String, mensaje: String): boolean
}

class "ProveedorCorreo" as ProvMail <<boundary>> {
+ enviarMensaje(destino: String, mensaje: String): boolean
}

Notificador <|-- NotifBasico
IProvCom <|.. ProvWA
IProvCom <|.. ProvMail
@enduml

```



Implementación en diagrama de clases de GCM:

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "UI_SistemaMantenimiento" as UI <<boundary>> {
    + mostrarPantallaLogin()
    + mostrarPantallaClientes()
    + mostrarPantallaTecnicos()
    + mostrarPantallaMantenimientos()
    + mostrarPantallaEstadisticas()
}

interface "IControladorPrincipal" as IControlador <<control>> {

```

```

+ validarCredenciales(user, pass): boolean
+ gestionarCRUDCliente(datos: DTO)
+ gestionarCRUDTecnico(datos: DTO)
+ registrarMantenimiento(datos: DTO): Resultado
+ generarEstadisticas(periodo: String, filtro: String): Reporte
}

```

```

class "ProxyControlador" as Proxy <<control>> {
- controladorReal: ControladorReal
- sesionActiva: Usuario
+ validarCredenciales(user, pass): boolean
+ gestionarCRUDCliente(datos: DTO)
+ gestionarCRUDTecnico(datos: DTO)
+ registrarMantenimiento(datos: DTO): Resultado
+ generarEstadisticas(periodo: String, filtro: String): Reporte
- verificarPermisos(operacion: String): boolean
}

```

```

class "ControladorReal" as CtrlReal <<control>> {
+ validarCredenciales(user, pass): boolean
+ gestionarCRUDCliente(datos: DTO)
+ gestionarCRUDTecnico(datos: DTO)
+ registrarMantenimiento(datos: DTO): Resultado
+ generarEstadisticas(periodo: String, filtro: String): Reporte
}

```

IControlador <|.. Proxy

IControlador <|.. CtrlReal

Proxy o--> CtrlReal : delega operaciones válidas

UI --> IControlador : envía solicitudes

```

abstract class "Usuario" as Usu <<entity>> {
- nombreCompleto: String
- correoElectronico: String
- numeroTelefonico: String
- contrasena: String
}

```

```

class "Cliente" as Cli <<entity>> {
    - nombreUsuario: String
    - cedula: String
}

class "Tecnico" as Tec <<entity>> {
    - especialidad: String
}

Usu <|-- Cli
Usu <|-- Tec

interface "IComponenteMantenimiento" as ICompMant <<entity>> {
    + obtenerCostoTotal(): double
    + obtenerCantidadEquipos(): int
}

class "Mantenimiento" as MantInd <<entity>> {
    .. Encabezado y Equipo ..
    - idMantenimiento: String
    - fechaRegistro: Date
    - equipo: String
    - modelo: String
    - marca: String
    - clavePin: String
    - numeroSerieImei: String
    - accesorios: String
    - tipoEquipo: String
    .. Daño a Reparar ..
    - enciende: boolean
    - botones: boolean
    - camara: boolean
    - sensores: boolean
    - touchId: boolean
    - wifi: boolean
    - senal: boolean
    - sonido: boolean
    - carga: boolean
}

```



```

.. Costos y Estado ..
- observaciones: String
- totalMantenimiento: double
- abono: double
- saldo: double
- fechaEstimadaEntrega: Date
- estado: String
--
+ obtenerCostoTotal(): double
+ obtenerCantidadEquipos(): int
}

class "GrupoMantenimientos" as GrpMant <<entity>> {
- mantenimientos: List<IComponenteMantenimiento>
- criterioAgrupacion: String
+ agregar(c: IComponenteMantenimiento)
+ remover(c: IComponenteMantenimiento)
+ obtenerCostoTotal(): double
+ obtenerCantidadEquipos(): int
}

ICompMant <|.. MantInd
ICompMant <|.. GrpMant
GrpMant o--> ICompMant : contiene

Cli "1" -- "0..*" ICompMant : solicita >
Tec "1" -- "0..*" ICompMant : repara >

CtrlReal --> Cli : instancia / valida
CtrlReal --> Tec : instancia / valida
CtrlReal --> ICompMant : procesa

class "RepositorioBaseDatos" as Repo <<repository>> {
+ guardarUsuario(u: Usuario): void
+ buscarClientePorCedula(ced: String): Cliente
+ guardarMantenimiento(m: IComponenteMantenimiento): void
+ obtenerVolumen(periodo: String): IComponenteMantenimiento
}

```

CtrlReal --> Repo : persiste datos

```
class "GestorNotificaciones" as Notificador <<control>> {  
  - proveedorAPI: IProveedorComunicaciones  
  + setProveedor(p: IProveedorComunicaciones)  
  + notificarCliente(celular_email: String, msj: String)  
}
```

```
interface "IProveedorComunicaciones" as IProvCom <<boundary>> {  
  + enviarMensaje(destino: String, mensaje: String): boolean  
}
```

```
class "ProveedorWhatsApp" as ProvWA <<boundary>> {  
  + enviarMensaje(destino: String, mensaje: String): boolean  
}
```

```
class "ProveedorCorreo" as ProvMail <<boundary>> {  
  + enviarMensaje(destino: String, mensaje: String): boolean  
}
```

Notificador o--> IProvCom : usa

IProvCom <|.. ProvWA

IProvCom <|.. ProvMail

CtrlReal --> Notificador : orquesta

@enduml

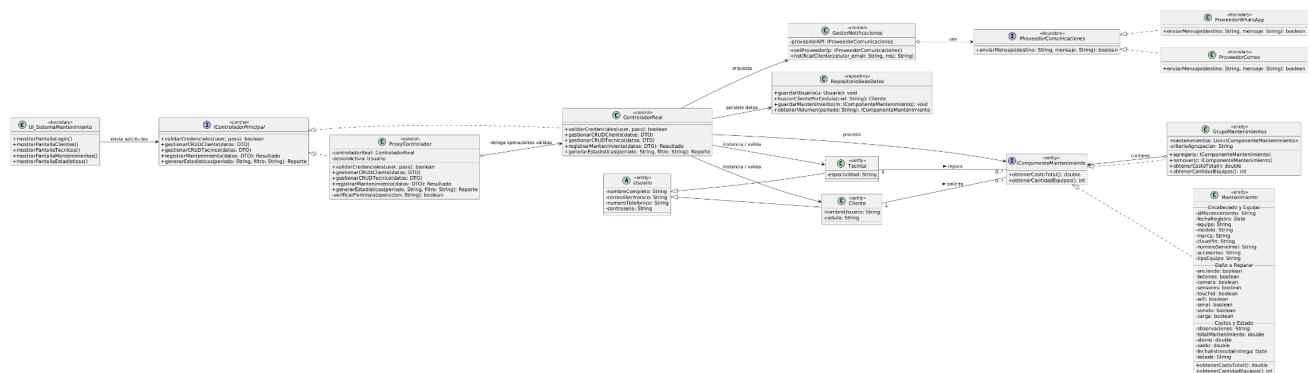


Diagrama de Arquitectura

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

package "Capa de Presentación" #E3F2FD {
    component "UI_SistemaMantenimiento" as VistaMantenimiento
}

package "Capa de Lógica de Negocio" #FFF3E0 {
    interface "IControladorMantenimiento" as ICM
    component "ProxyControlador \n (Proxy)" as Proxy #LightBlue
    component "ControladorMantenimientoReal \n (Servicio base)" as CMR

    component "NotificadorReportes \n (Bridge - Abstracción)" as AbstraccionBridge
    interface "IProveedorComunicaciones \n (Bridge - Implementación)" as IPC
    component "ProveedorWhatsApp" as PWA #LightGreen
    component "ProveedorCorreo" as PMC #LightGreen
}

package "Capa de Datos" #E8F5E9 {
    interface "IComponenteMantenimiento" as IComp #LightPink
    entity "MantenimientoIndividual \n (Hoja)" as MantInd #LightPink
    entity "GrupoMantenimientos \n (Compuesto)" as GrpMant #LightPink
    database "RepositorioGeneral" as Repo
}

' Relaciones entre capas y componentes
VistaMantenimiento --> ICM : envía solicitudes de gestión

Proxy ..|> ICM
CMR ..|> ICM
Proxy o--> CMR : delega operaciones autorizadas

CMR --> AbstraccionBridge : orquesta envío de reportes
AbstraccionBridge o--> IPC : usa puente estructural
PWA ..|> IPC
PMC ..|> IPC

CMR --> Repo : guardarMantenimiento(), \n obtenerVolumen()
Repo --> IComp : persiste y recupera jerarquías

```

MantInd ..|> IComp

GrpMant ..|> IComp

GrpMant o--> IComp : contiene

note top of Proxy

Proxy:

Intercepta las solicitudes de la UI,
verifica permisos según el actor en sesión
(Admin, Técnico o Cliente) y registra logs
de seguridad antes de acceder al servicio base.

end note

note top of AbstraccionBridge

Bridge:

Desacopla la lógica de negocio que define
qué reportes enviar (Abstracción) de los detalles
técnicos de conexión de las APIs masivas de
mensajería (Implementación: WhatsApp y Correo).

end note

note bottom of IComp

Composite:

Permite tratar de manera uniforme un
registro técnico individual y colecciones masivas
de datos en memoria (grupos por mes, fallos o marcas),
facilitando el cálculo fluido de estadísticas.

end note

legend right

|= Color |= Significado |

|<#E3F2FD> | Capa de Presentación |

|<#FFF3E0> | Capa de Negocio |

|<#E8F5E9> | Capa de Datos |

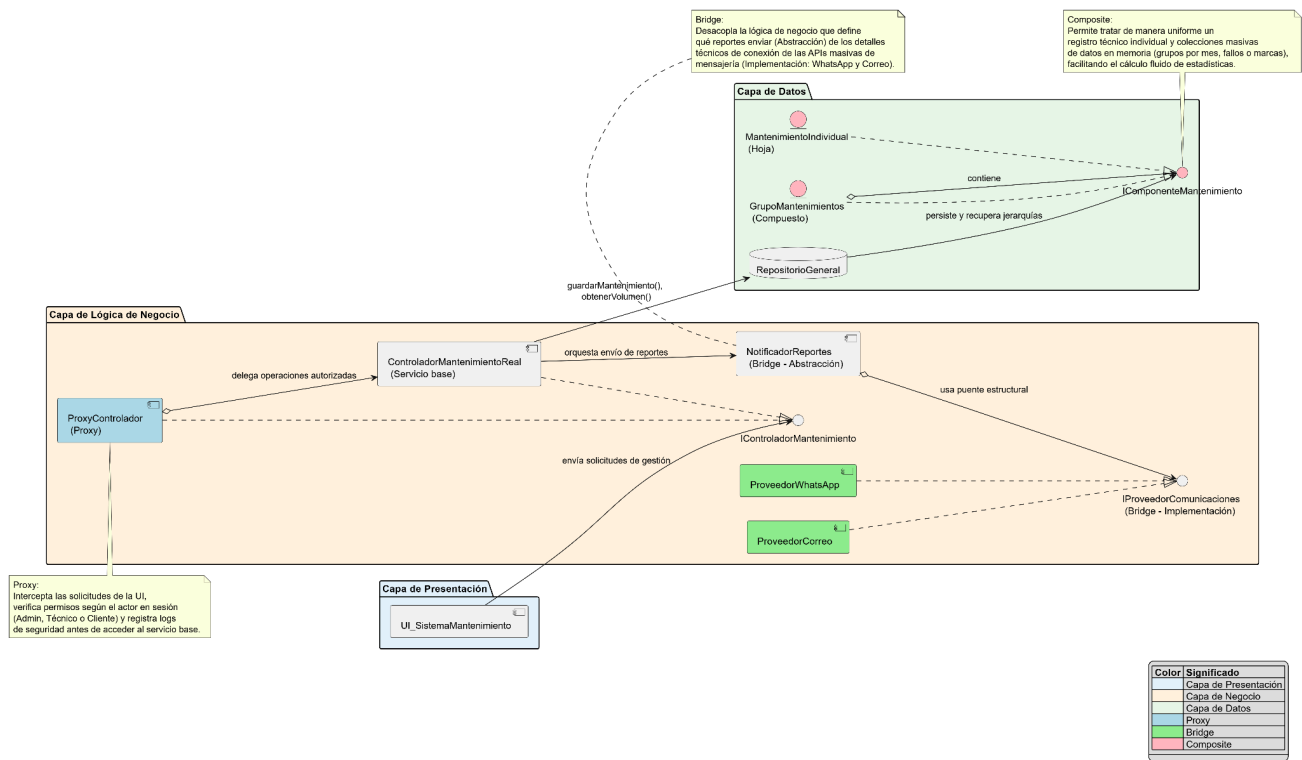
|<#LightBlue> | Proxy |

|<#LightGreen> | Bridge |

|<#LightPink> | Composite |

end legend

@enduml



CONCLUSIONES

La implementación conjunta de los patrones de diseño en este programa ha demostrado cómo una arquitectura bien planificada puede transformar el Sistema de Gestión Integral en uno altamente flexible y escalable. Al utilizar el patrón Composite, logramos unificar el tratamiento de mantenimientos individuales y grupos bajo una misma interfaz. Esto nos sirvió para simplificar enormemente la lógica del sistema, ya que el GestorCentral y el RepositorioGeneral pueden extraer datos masivos y consolidar métricas sin preocuparse de la jerarquía o nivel de agrupamiento de la información. Por su parte, el patrón Bridge nos permitió separar de forma limpia la orquestación de las notificaciones de los detalles técnicos de las APIs de WhatsApp y Correo, previniendo el acoplamiento y facilitando el mantenimiento.

Además, la integración del patrón Proxy añadió una capa de seguridad y auditoría transparente para el resto del sistema, sin necesidad de modificar el código de la lógica principal. En nuestro programa, el Proxy sirvió como un "guardia de seguridad" que intercepta las peticiones de la interfaz gráfica (UI_ModuloGestion), validando de forma estricta los permisos de los actores (Administrador, Cliente, Técnico) antes de permitir que el servicio real lea o altere la base de datos. En conjunto, la combinación de estos tres patrones nos sirvió para cumplir con el Principio de Responsabilidad Única, logrando un entorno de nivel empresarial robusto, modular y preparado para manejar de forma segura el control integral de las operaciones técnicas.

RECOMENDACIONES

Implementar un sistema real de roles utilizando el Proxy para mitigar brechas de seguridad: Actualmente, la inclusión de múltiples actores en casos de uso compartidos puede generar vulnerabilidades. Se recomienda extender la funcionalidad del ProxyGestorCentral inyectando el contexto del usuario en sesión (sus credenciales y su rol definido: Admin, Tec, Cli). De esta forma, se puede configurar el Proxy para que bloquee de raíz métodos críticos (como el de procesar estadísticas o asignar

técnicos) si el usuario no posee privilegios administrativos, aprovechando el verdadero potencial de seguridad de este patrón estructural.

Aprovechar el Bridge para implementar un manejo de errores robusto en las APIs externas: Dado que el sistema depende del consumo de servicios externos para la comunicación, la recomendación es usar las implementaciones concretas del patrón Bridge (ImplementadorWhatsApp e ImplementadorCorreo) para aislar la gestión de fallos. Se sugiere programar lógicas de reintentos (retries), manejo de tiempos de espera (timeouts) y captura de errores de autenticación directamente dentro de las clases concretas de la API, de manera que el GestorCentral nunca se congele si el proveedor de WhatsApp o de Correo experimenta una caída.

Sangolquí, a 09 de abril de 2026